

LANGCHAIN AND REACTIVE AGENTS

Module 3

Syllabus

LangChain fundamentals: Chains, Tools, Memory, Agents, ReAct pattern:

Reasoning + Acting framework, Building basic decision-making flows in theory,

Limitations and design challenges of reactive agents

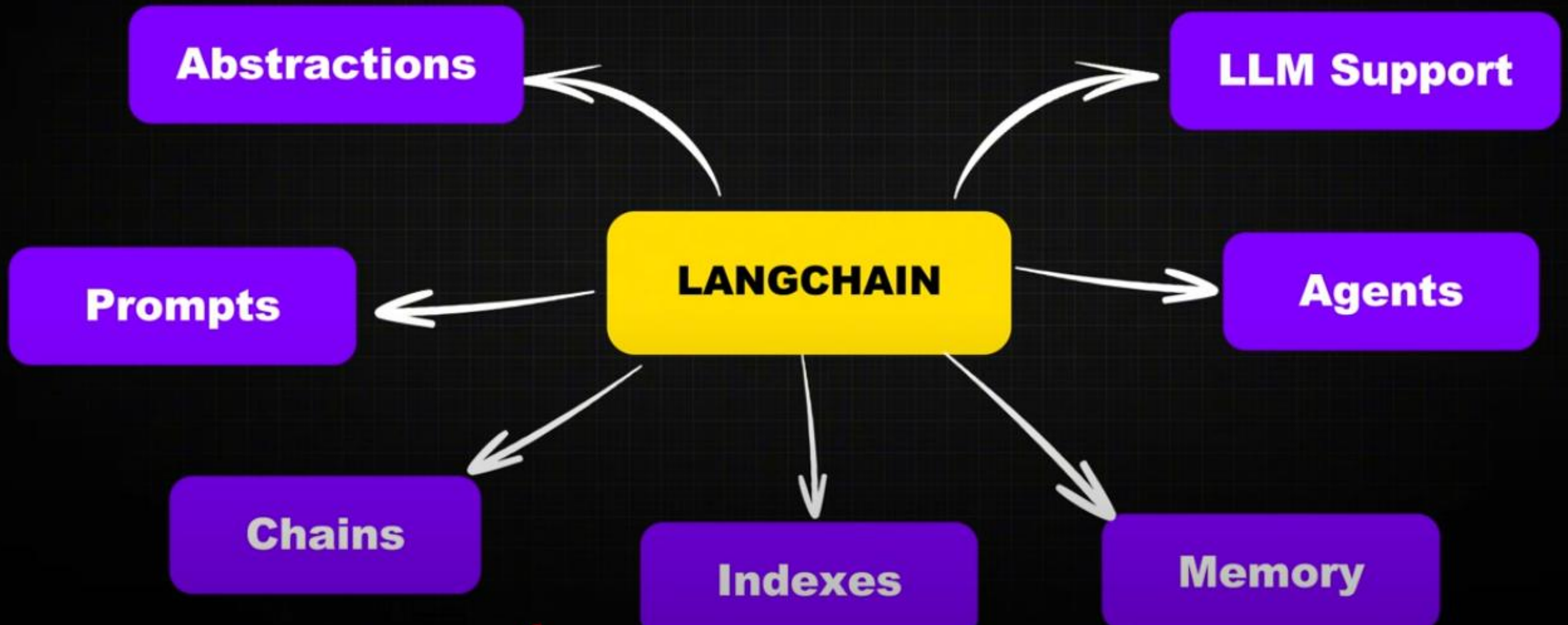
LangChain

- LangChain is one of the most powerful and widely adopted frameworks for building LLM-powered agents.
- It gives developers tools to **chain together prompts, models, memory, and external tools in logical workflows.**
- You can build everything from question-answering bots to document processors to autonomous agents.

Key Features:

- Native support for OpenAI, Anthropic, Cohere, Hugging Face, and local LLMs
- Rich ecosystem of tools: Google Search, SQL, Python execution, APIs, and more
- Supports both simple chains and complex agent loops (e.g., ReAct, Self-Ask)
- Memory and conversation tracking, including vector store integrations
- Highly extensible with custom tool and agent definitions

Langchain's Key Features



Overview

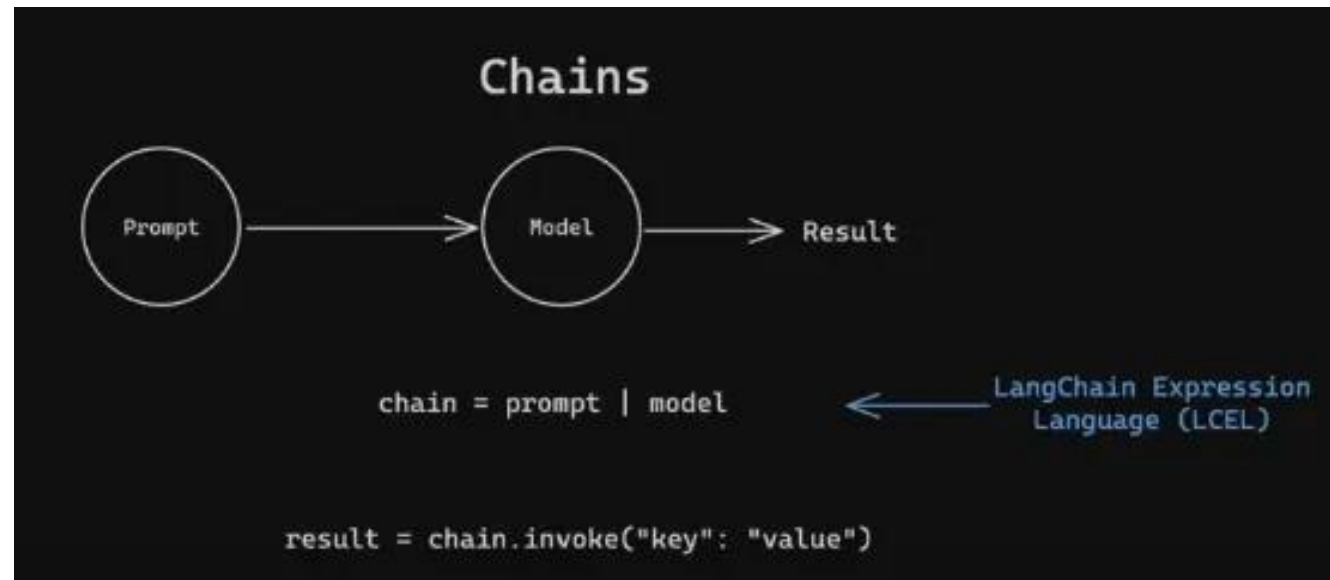
- LangChain is an open source framework with a pre-built agent architecture and integrations for any model or tool — so you can build agents that adapt as fast as the ecosystem evolves

Chain

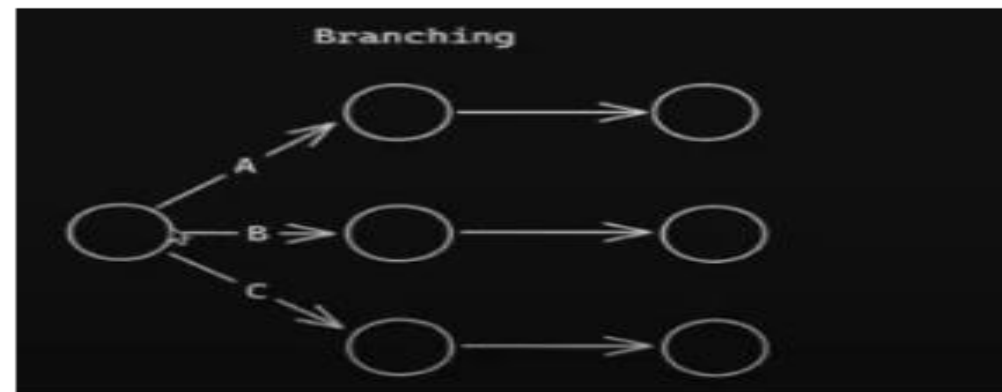
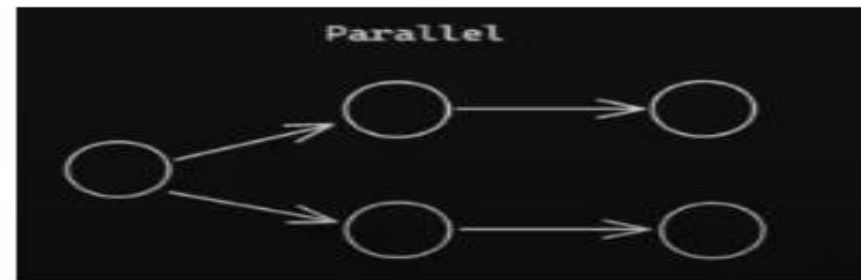
- In Lang Chain, a Chain is a sequence of modular components—such as Prompt Templates, Language Models (LLMs), and output parsers—linked together to perform a specific task.
- Chains allow you to create automated workflows where the output of one step becomes the input for the next

Chains

- LangChain provides various chain types that allow developers to build and customize workflows for natural language processing tasks.
- These chain types help streamline the integration of language models and other tools into applications.



Types of Chains



Chains

- Use LangChain Chains when your LLM application requires:
- Sequential steps (multi-stage reasoning)
- External data integration (APIs, databases, search)
- Stateful interactions (memory, user profiles)
- Complex business workflows

Types of chain

- LLMChain: The most basic chain that takes a user input, formats it using a prompt template, and passes it to an LLM for a response.
- Simple Sequential Chain: A linear pipeline where each step has a single input and single output.
- Sequential Chain: A more flexible version of the sequential chain that supports multiple inputs and outputs per step.
- Router Chain: Dynamically selects which specialized sub-chain to use based on the input (e.g., routing a math question to a "math chain" and a history question to a "history chain").
- Retrieval QA Chain: Specifically designed for Retrieval-Augmented Generation (RAG), it retrieves relevant documents from a database and uses them as context for the LLM to answer a question

Tools

- In LangChain, **tools are callable functions that allow Language Models (LLMs) to interact with the external world** (e.g., searching the web, accessing databases, running code, calling APIs).
- This capability extends the LLM's potential beyond static knowledge, enabling it to perform dynamic, real-world tasks through agents
- Tools extend what agents can do—letting them fetch real-time data, execute code, query external databases, and take actions in the world.
- Under the hood, tools are callable functions with well-defined inputs and outputs that get passed to a chat model.
- The model decides when to invoke a tool based on the conversation context, and what input arguments to provide.

Types of tools

- **Search Tools:** For fetching real-time information from the internet.
 - DuckDuckGo Search
 - [Google Search](#) (via APIs like SerpApi or Tavily)
 - Wikipedia API Wrapper
- **Code Interpretation:** For executing dynamic code in a sandboxed environment, useful for calculations or data processing.
 - Python REPL Tool (REPL stands for Read-Eval-Print Loop)
- **Data Access and APIs:** For interacting with external systems and data sources.
 - [Requests Tool](#) (for making HTTP requests to any API)
 - SQL Database Toolkit (for querying databases)
 - Gmail Toolkit (for reading and sending emails)
 - GitHub Toolkit (for managing repository interactions)
- **Web Browsing:** Tools that can interact with a web browser to scrape content or interact with web pages.
- **Utility Tools:** Simple, yet powerful tools for specific tasks.
 - [Calculator Tool](#) (for numerical operations)

Memory

- Memory is a system that remembers information about previous interactions.
- For AI agents, memory is crucial because it lets them remember previous interactions, learn from feedback, and adapt to user preferences.
- As agents tackle more complex tasks with numerous user interactions, this capability becomes essential for both efficiency and user satisfaction.

Types of Memory

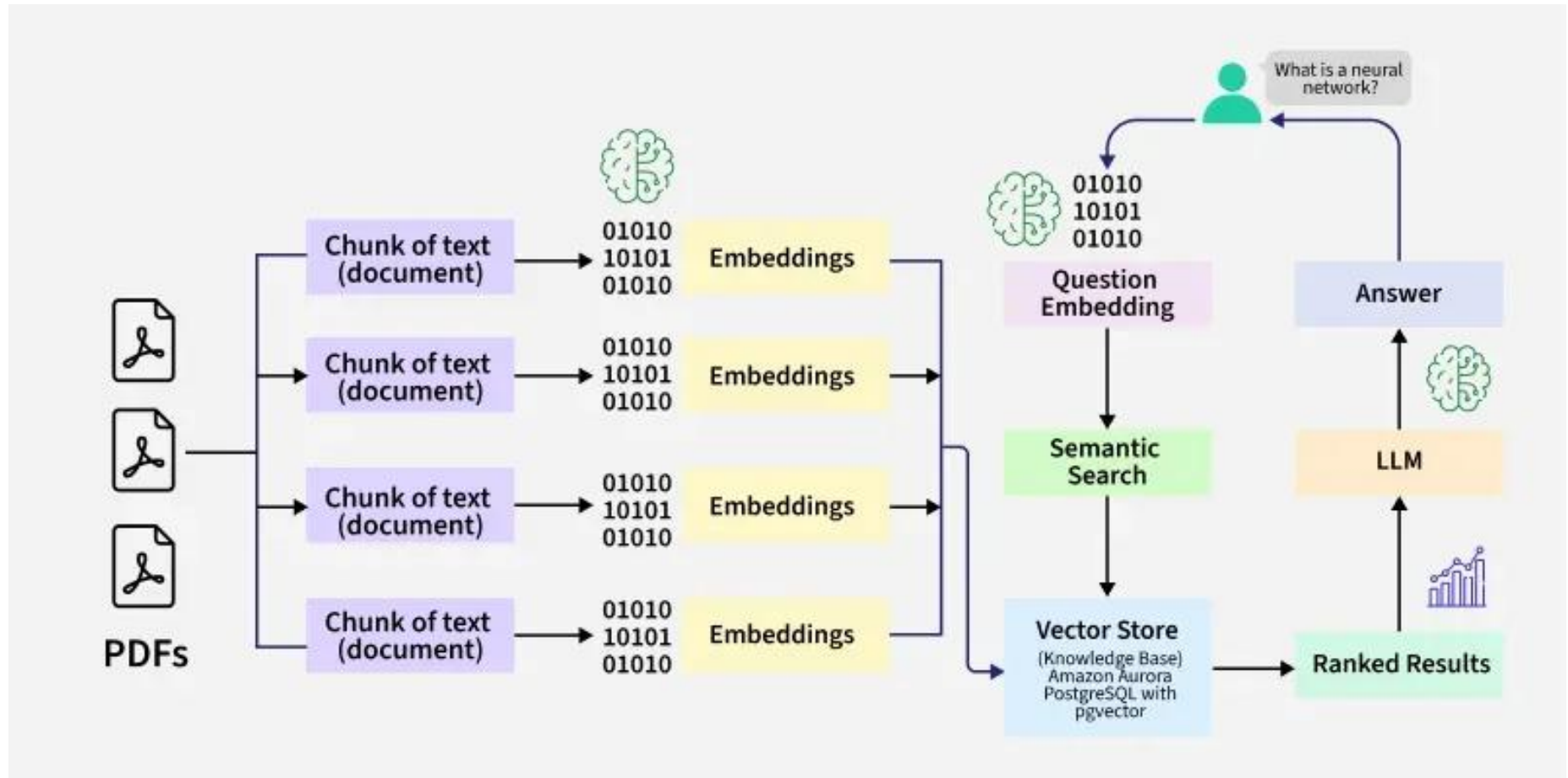
- Short-term memory, or thread-scoped memory, tracks the ongoing conversation by maintaining message history within a session.
- Long-term memory stores user-specific or application-level data across sessions and is shared across conversational threads. It can be recalled at any time and in any thread. Memories are scoped to any custom namespace, not just within a single thread ID.

Short-term memory

- Memory is a system that remembers information about previous interactions. For AI agents, memory is crucial because it lets them remember previous interactions, learn from feedback, and adapt to user preferences. As agents tackle more complex tasks with numerous user interactions, this capability becomes essential for both efficiency and user satisfaction.
- Short term memory lets your application remember previous interactions within a single thread or conversation.

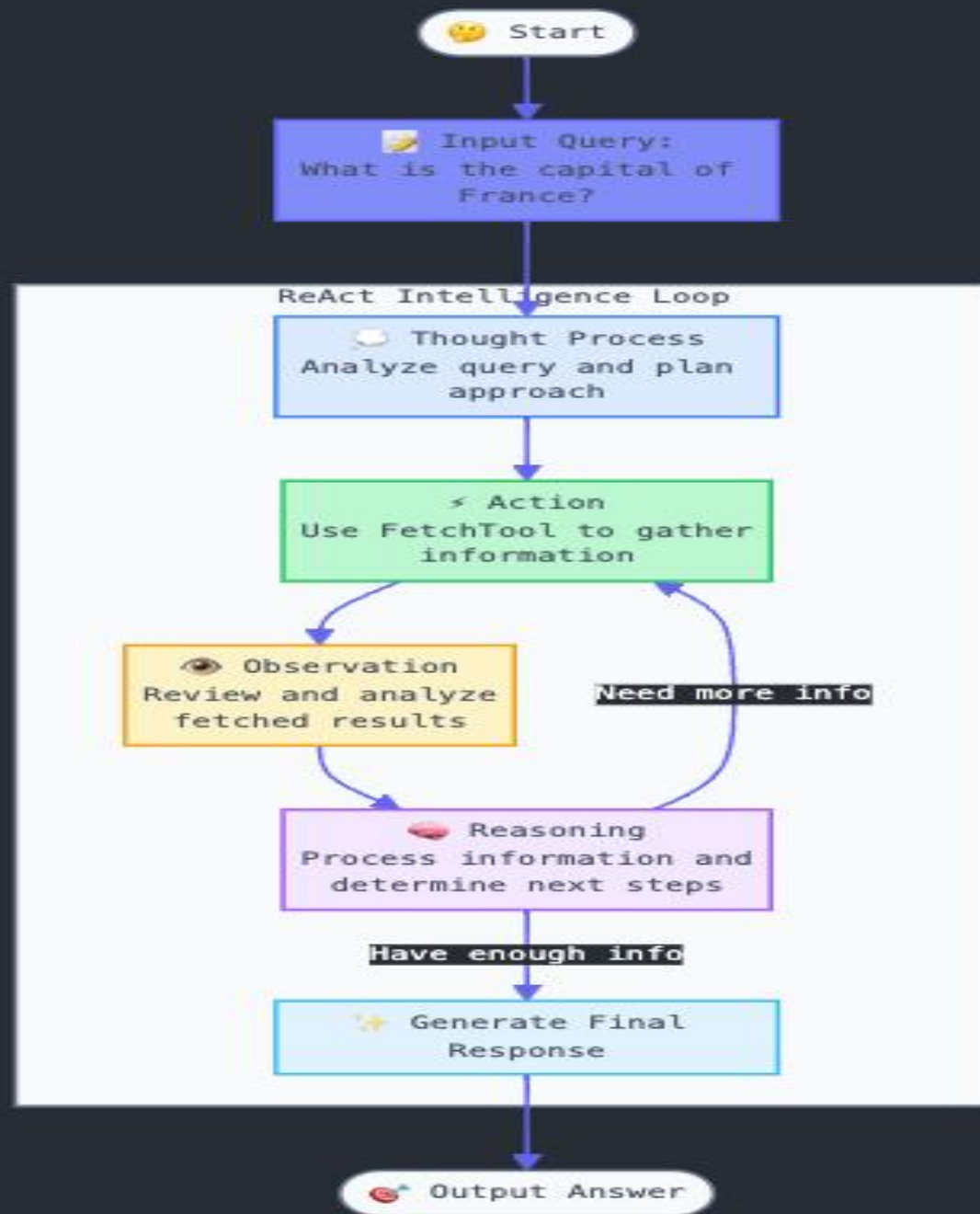
How LangChain Works?

- LangChain follows a structured pipeline that integrates user queries, data retrieval and response generation into seamless workflow.



ReAct pattern: Reasoning + Acting framework

- The **ReAct (Reasoning + Acting) pattern** is a prompting framework that enables Large Language Models (LLMs) to solve complex tasks by interleaving deliberate, step-by-step reasoning ("Thought") with specific actions ("Action") such as searching the web or using APIs. It enables agents to maintain, update, or adjust plans while interacting with external tools and observing results ("Observation"), which reduces hallucinations and increases accuracy compared to standard prompting



Core Components of ReAct:

Thought (Reasoning): The model explicitly verbalizes what it knows, identifies what it needs to find out, and breaks down the problem into smaller, manageable steps.

Action: The agent calls an external tool (e.g., search engine, database, calculator, or specialized API) to retrieve information or perform a task.

Observation (Results): The model processes the output from the tool and incorporates this new information into its next thought-action cycle.

Loop: This cycle of Thought → Action → Observation is repeated until the agent has sufficient information to provide a final, verified answer.

An LLM following ReAct generates reasoning traces (the “Thoughts”) and task-specific actions (calls to tools) in an interleaved manner. This means the model’s output might look something like:

Thought: I should calculate the total.

Action: Calculator("123 + 456")

Observation: 579

Thought: Now I have the sum; next, I need to multiply it.

Action: Calculator("579 * 789")

Observation: 456,831.

Thought: I have the final result.

Final Answer: 456,831.

The reasoning traces (the chain of thoughts) help the model plan and keep track of what to do next, while the actions let it consult external sources or perform calculations to gather information it otherwise wouldn’t have inherent access to.

In the example shown below, we see a live trace of an AI News Reporter Agent executing a task using the ReAct paradigm. The agent has been asked to create a news headline related to the "Agent2Agent Protocol"

Agent Role: News Reporter – This agent specializes in curating and validating news content.

Task: The agent has been instructed to generate a news headline and ensure it's relevant to the Agent2Agent Protocol.

Thought: The agent first reasons internally that it should validate the information by performing a quick search for any recent updates about the protocol. This is the reasoning part of the ReAct cycle.

Action: It proceeds to use a tool labeled Search the internet, passing in a structured JSON input with the query: "Agent2Agent Protocol news November 2023". This is the acting step where the agent leverages external tools to gather real-world data.

Building basic decision-making flows in theory,

A basic decision-making flow consists of:

- **Input** → User query or data
- **Processing/Reasoning** → Analyze the input
- **Decision** → Choose an appropriate action
- **Output** → Final response or result

2. Components of Decision Flow

- **Input Handler:** Receives user request
- **Decision Logic:** Applies rules or reasoning
- **Action Module:** Executes the chosen task
- **Output Generator:** Produces the final result

Simple Flow Structure

1. User gives input
2. System analyzes the input
3. Decision is made (based on rules or conditions)
4. Appropriate action is performed
5. Output is returned

Example (Real-World)

Customer Support System:

- User: *"I want to return my product"*
- System checks:
 - If order exists → proceed
 - If within return period → approve
- Action: Initiate return process
- Output: Confirmation message

Types of Decision Flows

- **Rule-based flow:** Uses if-else conditions
- **Sequential flow:** Steps executed one after another
- **Conditional flow:** Different paths based on decisions

Reactive agents operate on direct stimulus-response (if-then) rules, making them fast and efficient in static environments, but severely limited by a lack of memory, learning, and long-term planning.

Key design challenges include handling partially observable environments, managing unexpected scenarios, and preventing suboptimal behavior in complex tasks

Limitations of Reactive Agents

Lack of Memory/Internal State: They cannot store past experiences, meaning they cannot learn or adapt over time, and they forget information quickly.

Suboptimal Decision-Making: Relying solely on current sensory input restricts their ability to solve complex, long-term problems, often resulting in inferior outcomes.

Limited Adaptability: They struggle in dynamic, unpredictable, or partially observable environments, as their predefined rules cannot handle unforeseen scenarios.

Hardcoded Behavior: They require precise, predefined mapping of conditions to actions, which becomes impractical in complex environments.

Design Challenges

Environmental Complexity: Designing for partial observability means the agent cannot see the whole environment, making it hard to act correctly based only on local info.

Rule Engineering: Developing accurate "if-then" rules to cover all necessary scenarios without causing contradictory actions is challenging.

Scalability: While simple for small tasks, increasing the number of rules for complex tasks leads to maintenance bottlenecks.

Predictability and Testing: While they are predictable, debugging to find which rule failed in a chain of reactions can be difficult.

Reactivity vs. Goal-Orientation: Balancing rapid reaction times with the need to achieve long-term goals can be tricky, as they often lack long-term planning mechanisms

1. Poor Data Quality & Inconsistent Access

This one hits early. You can't build a smart agent without data, but good data is surprisingly hard to find. A lot of the time, the data you *do* get is:

- Unstructured (like messy text or audio logs),
- Full of noise (think typos, broken logs, incomplete records), or
- Spread across different systems that don't talk to each other.

Algorithmic & Systemic Bias

Even if your data looks clean, there's another trap: bias. It creeps in silently.

Let's say your customer service agent was trained mostly on interactions from English speakers in one region. That might seem fine... until someone with a different accent or background talks to it, and it gives a weird or offensive response. Or worse, ignores them completely.

Bias can show up in how data was collected, how it was labeled, or even in the assumptions baked into the model itself. And once it's there, the agent will just keep repeating it. Identifying and fixing this isn't easy, but if you ignore it, it can lead to serious trust and reputation issues.

3. Understanding Context & Intent

This one's a biggie, especially for agents that deal with language.

Humans speak with tone, sarcasm, shorthand, and often without saying exactly what they mean. An agent might hear:

“Wow, that’s just great.”

... and think it's a compliment. But we all know that tone probably means the opposite.

Multi-turn conversations (where users ask follow-up questions or switch topics) are even tougher. The agent needs to remember what was said earlier, understand how it relates to what's being asked now, and respond in a way that actually makes sense.

Integration with Legacy or Existing Systems

This might be the least exciting part, but it's where so many projects stall.

You can build the smartest agent in the world, but if it needs to pull customer data from a 15-year-old CRM or trigger actions in a legacy ERP system, good luck. Many of these systems don't have proper APIs, or they rely on outdated formats that modern tools don't play nicely with.

So what happens? Teams either spend months writing custom connectors, or worse, they force the agent to work in a very limited sandbox, which kills a lot of its usefulness.